

DBMS Assignment II

Set 4

Q2) Consider the following relations: Sailors (sid, sname, rating, age) Boats (bid, bname, color) Reserves (sid, bid, day). Write the SQL statements for the following questions.

- find the names of sailors who have reserved a red boat.
- find the names of sailors who have reserved at least one boat.
- find the names of sailors who have reserved a Red and a Green boat.
- find the names of sailors who have reserved a Red or a white boat.
- find the names of sailors who have reserved all boats.

A) a) SQL code:

```
select distinct S.sname from Sailors S, Reserves  
R, Boats where S.sid = R.sid and R.bid = B.bid  
and B.color = 'red';
```

Explanation: Joins all three tables using their matching keys (sid and bid). It implicitly restricts rows to those where the boat record color maps exactly to 'red' and projects distinct sailors names to eliminate operational duplicates.

b) find the names of sailors who have reserved at least one boat.

SPL code: select distinct S.sname from sailors S,
Reserves R where S.sid = R.sid;

Explanation: performs a relational join directly b/w sailors entity table and the Reserves active transaction logging table on 'sid'. The simple existential footprint of a sailor's ID in the transaction map marks validation for having a minimum reservation count of one.

c) find the names of sailors who have reserved a Red and Green boat.

SPL code: select S.sname from sailors S, Reserves R,
Boats B where S.sid and R.bid = B.bid and B.color =
'red'

intersect

select S2.sname from sailors S2, Reserves R2, Boats B2
where S2.sid = R2.sid and R2.bid = B2.bid and B2.color =

Explanation: Because a single row evaluation can't yield a boat color value that matches both 'red' and 'green' simultaneously, standard conjunction drops valid results. The intersect set operation safely forces structural evaluation

208 of two isolated queries - isolated subsets of red boat selectors and green boat selectors - returning only intercept entities.

d) find the names of sailors who have reserved a red or a white boat.

SQL code: Select distinct S.sname from Sailors S,
Reserves R, Boats B where S.sid = R.sid and R.bid = B.bid
and (B.color = 'red' OR B.color = 'white');

Explanation: combines logical sets using an explicit parenthetical disjunction clause (OR). This matches any operational entries matching either attribute criterion independently, or capturing overlap rows matching both conditions structurally.

e) find the names of sailors who have reserved all boats.

SQL code: Select S.sname from Sailors S where not
exists (select B.bid from Boats B where not exists
(select R.bid from Reserves R where R.bid = B.bid and
R.sid = S.sid));

Explanation: Employs relational division mapped through a double-nested 'NOT EXISTS' block structure. Semantically parsed, It retrieves target profiles by determining that

There doesn't exist any instance of a boat inside the Boats master register that possesses no corresponding transaction cross-reference matching the candidate earlier record inside the transaction logging table!

4Q) List the conditions required for a relation to be in 4NF and 3NF explain with examples.

A) $\Rightarrow$ Third Normal Form (3NF).

Definition: A structural relation is properly validated under third Normal form criteria if it satisfies standard second normal form (2NF) prerequisites and contains non-prime attributes demonstrating a transitive structural dependency on any candidate key framework.

for every non-trivial functional dependency condition generalized as $X \rightarrow A$, at least one of the following constraints must be explicitly met:

- $\rightarrow$ X is a valid super key structure within the relation, OR
- $\rightarrow$ A is verified as a prime attribute (a member of a primary candidate key mapping structure).

Scenario Illustration: consider the un-decomposed composite relation tracking employees and their operational station assignments: Emp-Dept (EmpID, DeptID, DeptName).

Anomalous Impact: Dependencies: $\text{EmpID} \rightarrow \text{DeptID}$,
 and $\text{DeptID} \rightarrow \text{DeptName}$. Because DeptID functionally
 determines DeptName while falling violations as a
 structural super key, a cross-transitive dependency violation
 exists ($\text{EmpID} \rightarrow \text{DeptName}$). This causes severe anomaly
 issues; if all employees leave a department, the physical
 mapping of the department ID to its corporate internal
 name is entirely deleted.

Structural Representation: Table state: Violating 3NF (Emp-Dept)

EmpID (Key Component)	DeptID	DeptName
E001	D10	Human Resources
E002	D20	Engineering
E003	D10	Human Resources.

Normalization Resolution: Decomposition solution: Resolution
 requires normalizing the relation by cleanly splitting
 the dataspace into two related atomic structures: an
 Employee cross-reference matrix and an autonomous
 Department context entity matrix.

Table Framework A: Decomposed Normalization

step1: Employee key Matrix table.

EmpID (primary key)	DeptID
E001	D10
E002	D20
E003	D10

Table Framework B: Decomposed Normalization
step2: Department context table.

DeptID (primary key)	DeptName
D10	Human Resources
D20	Engineering

2) Fourth Normal Form (4NF)

Definition: A relation satisfies the criteria for Fourth Normal Form validation if and only if it sits conformant within standard Boyce-Codd Normal Form (BCNF) specifications, and doesn't contains any Independent Multi valued Dependencies (MVDs).

Formally, for every non-trivial structural Multi-Valued Dependency mapped as $X \twoheadrightarrow Y$ the determining determinant attribute X must possess verified identity as a super key framework within the database model.

Scenario Illustration: Consider an un-decomposed relation tracking academic course allocation components. $\text{Course}(\text{CourseName}, \text{professor}, \text{Textbook})$, given structural assumption that a single distinct course name support multiple independent instructors alongside multiple independent textbooks.

Anomalous Impact: This rule definition produces multivalued dependencies $\text{CourseName} \twoheadrightarrow \text{professor}$ and $\text{CourseName} \twoheadrightarrow \text{Textbook}$.

Because CourseName alone lacks proper validation standing as a unique structural super key, it severely violates 4NF rules. This failure induces redundancy anomalies. If an instructor assigns an updated textbook to a class, a manual combinatorial loop must force the injection of multiple redundant rows to clearly map every course product patterns.

structural Representation: Table state: Violating 4NF (2)

CourseName (Composite key)	professor	Textbook
CS101	Dr Smith	Intro to DB
CS101	Dr Smith	Advanced SQL
CS101	prof. Davis	Intro to DB
CS101	prof. Davis	Advanced SQL

Normalization Resolution: Decomposition solution: Decompose the entity space into two independent binary projections that cleanly segment the unrelated multivalued factual dimension.

Table Framework A: Decomposed Normalization step: Course to faculty Map table.

CourseName	professor
CS101	Dr Smith
CS101	prof. Davis

Framework Table B

Step 2: Course to Literature Map table.

CourseName	Textbook
CS101	Intro to DB
CS101	Advanced SQL

11) Explain the checkpoint log based recovery scheme for recovering the database.

A checkpoint is a mechanism used by a DBMS to limit the amount of work it has to do during database recovery after a system crash.

1) The checkpoint operation

→ Memory Flush phase: System dynamically forces all log record held within main memory volatile buffers into disk blocks.

→ Dirty page Write phase: Automatically forces all active modified database block contents (dirty pages) residing inside volatile buffer blocks out into permanent physical file sectors.

→ Log Record Appending: Emits an absolute log boundary indicator marking formatted as <checkpoint L>, embedding a complete transaction list 'L' explicitly highlighting active transactions during the exact timeline threshold.

2) Recovery process After a Crash.

The automated recovery protocol passes the log stream backward to detect the absolute newest <checkpoint L> marker. It instantly generates an internal Undo-List (pre-populated using transaction IDs inside active recovery

List L) alongside a completely empty Redo-List framework.

The system next scans completely forward along the logging chain.

→ If a $\langle T \text{ start} \rangle$ structural block pattern is encountered, transaction entry T is instantly added to the active Undo-List.

→ If a $\langle T \text{ commit} \rangle$ or a $\langle T \text{ abort} \rangle$ structural pattern is encountered, transaction entry T is cleanly transitioned out of the Undo-List and placed safely into the active Redo-List container.

Execution steps:

Undo phase: processes historical logs backward from the crash tail to revert all partial adjustment made by lingering transactions left side the Undo-List.

Redo phase: processes historical logs entirely forward from the checkpoint boundary to safely rewrite and apply all committed actions executed by validated transactions stored inside the Redo-List.

Q1. Analyze the strengths and weaknesses of different file organizations used in database systems, such as sequential, indexed sequential and hashed files.

A) → Sequential File Organization: Records are structurally written and mapped sequentially inside continuous logical file tracks governed by a dedicated indexing search key. The actual physical hardware arrangement closely mirrors this logical progression layout.

→ Indexed Sequential File Organization: Records are structurally organized inside sequential pattern to enable lightning-fast sequential sweep operations; however, a dedicating multi-tier primary tree block index (such as an ISAM or B+ Tree) is constructed above the base data to map index pointers directly to individual data sectors for random record lookups.

→ Hashed File Organization: system implements an algorithmic hash conversion equation mapped across designated search by key values to calculate the precise physical hardware storage addresses or block coordinate where data rows must be read or saved.

File organization	Strengths	Weaknesses	Best used for	Primary operations cost
Sequential	Fast processing for complete sequence; zero storage overhead	Extremely slow individual point searches ($O(N)$); updates requires expensive physical block shifting	High volume batch operations (eg., corporate payroll files)	→ search: $O(N)$ → insert: $O(N)$ → delete: $O(N)$
Indexed Sequential	Accelerated direct point seeking ($O(\log N)$) combined with rapid sequential access paths.	Requires extra allocated storage space for backing index records; higher execution cost on updates.	Applications needing mixed lookups and range scans	→ search: $O(\log N)$ → insert: $O(\log N)$ → delete: $O(\log N)$
Hashed	Unrelated single record location and retrieval rates optimizing at $O(1)$ time complexity.	Highly inefficient for range scanning requests; vulnerable to performance decay if a collisions occurs.	Direct key-value lookups user access controls, and unique item lookups.	→ search: $O(1)$ Avg → insert: $O(1)$ Avg → delete: $O(1)$ Avg